

90-Day SRE Skill Reconstruction Path

Expert Panel Methodology

Curriculum Designer: This plan is reverse-engineered from how effective SREs actually learned. Each week unlocks concrete capabilities. No week is pure theory without immediate practical application.

Staff SRE: Every deliverable maps to real interview questions and on-call scenarios. By week 13, you should be indistinguishable from a junior SRE with 6 months of production experience.

Hiring Manager: This plan prepares you for both phone screens and on-site interviews. By day 14, you can pass basic screens. By day 90, you can handle technical deep-dives.

14-Day Checkpoint: First Concrete Capability

By the end of Week 2, you **MUST** be able to:

1. Set up Prometheus + Grafana from scratch
2. Instrument a service with RED metrics
3. Define and measure an SLI/SLO
4. Detect, diagnose, and document a simulated production failure
5. Write a basic post-mortem

If you cannot do these five things, you're not ready to proceed. Repeat Weeks 1-2.

Week-by-Week Breakdown

WEEK 1: Observability Foundations I

Goal: Understand what "observable" means and set up your first monitoring stack

Day 1: Mental Models

- **Theory** (2 hours):
- Read: Google SRE Book, Chapter 6 "Monitoring Distributed Systems"

- Read: "The Four Golden Signals"
- Understand: Metrics vs Logs vs Traces

- **Deliverable:** Write 1-page summary:
 - What are the four golden signals?
 - When would you use metrics vs logs?
 - What's the difference between latency and saturation?

Day 2: Prometheus Setup

- **Hands-on** (4 hours):
 - Install Prometheus + Grafana via Docker Compose
 - Deploy sample app (any HTTP API)
 - Configure Prometheus to scrape sample app
 - Verify metrics appear in Prometheus UI
- **Deliverable:** Screenshot of Prometheus UI showing scraped metrics

- **Resources:**

```
```bash
docker-compose.yml
version: '3'
services:
 prometheus:
 image: prom/prometheus:latest
 ports:
 - "9090:9090"
 volumes:
 - ./prometheus.yml:/etc/prometheus/prometheus.yml

 grafana:
 image: grafana/grafana:latest
 ports:
 - "3000:3000"
```
```

Day 3: Metrics Implementation

- **Hands-on** (4 hours):
 - Add Prometheus client to your app (Python/Go/Java)
 - Implement:
 - Counter: `http_requests_total{method, endpoint, status}`
 - Histogram: `http_request_duration_seconds`

- Expose `/metrics` endpoint
- Verify Prometheus scrapes metrics

- **Deliverable:** Working `/metrics` endpoint returning Prometheus format

- **Code Example** (Python Flask):

```
```python
from prometheus_client import Counter, Histogram, generate_latest

REQUEST_COUNT = Counter('http_requests_total', 'Total requests', ['method', 'endpoint',
'status'])
REQUEST_LATENCY = Histogram('http_request_duration_seconds', 'Request latency',
['endpoint'])

@app.route('/metrics')
def metrics():
 return generate_latest()

@app.before_request
def before_request():
 g.start_time = time.time()

@app.after_request
def after_request(response):
 latency = time.time() - g.start_time
 REQUEST_COUNT.labels(request.method, request.path, response.status_code).inc()
 REQUEST_LATENCY.labels(request.path).observe(latency)
 return response
```
```

Day 4: PromQL Basics

- **Learning** (3 hours):
- PromQL query types: instant vs range
- Functions: `rate()`, `sum()`, `avg()`, `histogram_quantile()`
- Practice queries in Prometheus UI

- **Exercises:**

```
```promql
Request rate (req/s)
rate(http_requests_total[5m])

Error rate (%)
rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m]) * 100
```
```

```
# P99 latency
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))
```
```

- **Deliverable:** Screenshot of 3 working PromQL queries

## Day 5: Grafana Dashboards

- **Hands-on** (4 hours):
- Create Grafana dashboard with:
- Request rate panel (line chart)
- Error rate panel (line chart)
- P50/P95/P99 latency (line chart, multi-line)
- Status code distribution (bar chart)

- **Deliverable:** Exported dashboard JSON

- **Dashboard Structure:**

```

Row 1: Traffic

- Total requests per second
- Requests by endpoint

Row 2: Errors

- Error rate percentage
- Errors by status code

Row 3: Latency

- P50/P95/P99 latency
- Latency heatmap

```

## Day 6: Load Testing

- **Hands-on** (4 hours):
- Install load testing tool: `hey` , `wrk` , or `locust`
- Generate load: 100 req/s for 5 minutes
- Observe metrics in Grafana in real-time
- Increase load to 500 req/s, observe changes

- **Deliverable:** Screenshot of dashboard under load

- **Commands:**

```bash

Using hey

hey -z 5m -q 100 http://localhost:8080/api/endpoint

```
# Using wrk
wrk -t4 -c100 -d5m http://localhost:8080/api/endpoint
...
```

Day 7: First Failure Detection

- **Hands-on** (4 hours):
 - Inject failure in your app:
 - Option 1: Add artificial latency
 - Option 2: Return 500 errors for 10% of requests
 - Observe in Grafana: Which golden signal detected it first?
 - Time yourself: How long to detect failure?
- **Deliverable:**
 - Written report: Failure injected, time to detection, which metric caught it
 - Screenshot of metrics during failure

Week 1 Checkpoint:

- Can you set up Prometheus + Grafana?
- Can you instrument a service with metrics?
- Can you write PromQL queries?
- Can you detect a failure in metrics?

If NO to any: Repeat Week 1

WEEK 2: Observability Foundations II + SLI/SLO

Goal: Define reliability targets and detect SLO violations

Day 8: SLI/SLO Theory

- **Theory** (3 hours):
 - Read: Google SRE Book, Chapter 4 "Service Level Objectives"
 - Read: "SLO vs SLA vs SLI"
 - Understand: Error budgets
- **Deliverable:** Write answers to:
 - What's the difference between SLI, SLO, and SLA?
 - How do you calculate error budget?
 - Why is 99.99% not always better than 99.9%?

Day 9: Define Your First SLI/SLO

- **Hands-on** (4 hours):
 - Choose your service's critical user journey

- Define SLI (e.g., "Latency of GET /checkout < 500ms")
- Choose SLO threshold (e.g., 99.9%)
- Calculate error budget (43.2 minutes/month for 99.9%)

- **Deliverable:** Document with:
 - SLI definition
 - SLO threshold and justification
 - Error budget calculation
 - Measurement method (PromQL query)

Day 10: SLO Measurement

- **Hands-on** (4 hours):
- Implement SLO measurement in Prometheus
- Create Grafana panel showing:
 - Current SLO compliance (%)
 - Error budget consumed (%)
 - Error budget remaining (minutes)

- **PromQL Example:**

```
```promql
SLO compliance (%)
(
 sum(rate(http_request_duration_seconds_bucket{le="0.5"}[30d]))
 /
 sum(rate(http_request_duration_seconds_count[30d]))
) * 100
```

# Error budget consumed (%)

```
(
 1 - (
 sum(rate(http_request_duration_seconds_bucket{le="0.5"}[30d]))
 /
 sum(rate(http_request_duration_seconds_count[30d]))
)
) / (1 - 0.999) * 100
```
```

- **Deliverable:** Grafana dashboard with SLO tracking

Day 11: Alerting Basics

- **Hands-on** (4 hours):
- Configure Prometheus Alertmanager

- Create alert rule:

```
yaml groups: - name: slo_alerts rules: - alert: SLOViolation expr: |
(
    rate(http_requests_total{status=~"5.."}[5m])
    /
    rate(http_requests_total[5m])
) > 0.05 for: 5m annotations: summary: "Error
rate above 5%"
```

- Test: Inject errors, verify alert fires

- **Deliverable:** Alert fires and you receive notification

Day 12: Simulate Production Incident

- **Hands-on** (6 hours):

- Scenario: Deploy buggy code that causes errors

- Timeline:

1. Deploy (introduce failure)
2. Detect (alert fires)
3. Triage (identify error spike in metrics)
4. Mitigate (rollback)
5. Document (write post-mortem)

- **Deliverable:**

- Post-mortem document:

- Summary
- Timeline with timestamps
- Root cause
- Impact (SLO violation, error budget consumed)
- Action items

Day 13: Error Budget Tracking

- **Hands-on** (4 hours):

- Calculate error budget at start of month: 43.2 minutes

- Simulate incidents throughout month:

- Incident 1: 15 minutes downtime

- Incident 2: 10 minutes degradation

- Track error budget consumed

- Decide: Can you launch new feature with 18.2 minutes remaining?

- **Deliverable:** Error budget spreadsheet or dashboard showing:

- Total budget
- Consumed by each incident
- Remaining

- Projection: Will budget last until month end?

Day 14: 14-Day Checkpoint Assessment

- **Hands-on** (4 hours):
- Given: Unfamiliar service (deploy a new sample app)
- Task:
 1. Instrument with Prometheus metrics (1 hour)
 2. Define SLI/SLO (30 min)
 3. Create Grafana dashboard (1 hour)
 4. Inject failure, detect, document (1.5 hours)

- **Success Criteria:**

- Complete all tasks in 4 hours
- Post-mortem is clear and actionable
- SLO definition is reasonable

Week 2 Checkpoint:

- Can you define SLI/SLO for a service?
- Can you measure SLO compliance?
- Can you detect and document a failure?

If NO to any: Repeat Week 2

If YES to all: You're ready for Week 3

WEEK 3-4: Incident Response Fundamentals

Goal: Learn incident triage, mitigation strategies, and communication

Day 15-17: Incident Response Theory

- Read: Google SRE Book, Chapter 14 "Managing Incidents"
- Study:
 - Severity classification (SEV1/2/3)
 - Incident roles (Commander, Comms, Ops)
 - Mitigation vs remediation

- **Deliverable:** Incident response runbook template

Day 18-20: Mitigation Strategies

- Practice each mitigation technique:
 1. Rollback (fastest)
 2. Feature flag disable
 3. Circuit breaker

4. Rate limiting
5. Scale up
6. Failover

- **Lab:** Implement each, measure time to mitigation

Day 21-24: Realistic Incident Simulations

- **Scenarios** (2 hours each):

1. Database connection pool exhausted
2. Downstream service timeout
3. Memory leak under load
4. Cache stampede
5. Cascading failure
6. Partial network partition

- For each:
 - Detect (how long?)
 - Triage (what's the root cause?)
 - Mitigate (what's the fastest fix?)
 - Document (post-mortem)

Day 25-28: Communication & Post-Mortems

- Practice writing:
- Incident updates (during incident)
- Post-mortems (after incident)
- Blameless language
- Actionable follow-ups

- **Exercise:** Given incident timeline, write post-mortem

Week 3-4 Deliverable: Portfolio of 6 post-mortems from simulated incidents

WEEK 5-6: Reliability Patterns & Failure Modes

Goal: Implement retry logic, circuit breakers, and understand distributed failures

Day 29-31: Retry Logic

- Implement:
- Exponential backoff
- Jitter

- Retry budget
- Idempotency

- **Lab:** Measure retry amplification

Day 32-35: Circuit Breakers

- Implement circuit breaker state machine
- Tune thresholds
- Test state transitions

- **Lab:** Prevent cascade failure with circuit breaker

Day 36-42: Distributed Systems Failures

- Study:
- Partial failures
- Thundering herd
- Split brain
- Cascade failures

- **Labs:**

- Simulate each failure mode
- Measure blast radius
- Implement mitigations

Week 5-6 Deliverable: Working implementations of retry logic and circuit breaker

WEEK 7-8: Distributed Systems Complexity

Goal: Multi-service dependencies, tracing, and capacity planning

Day 43-46: Distributed Tracing

- Set up Jaeger/Zipkin
- Implement trace context propagation
- Correlate logs across services

- **Lab:** Debug request through 5-service chain using traces

Day 47-50: Capacity Planning

- Learn:
- Utilization vs saturation

- Headroom calculation
- Load testing methodology

- **Lab:** Find capacity limit, calculate headroom

Day 51-56: Multi-Service Scenarios

- **Complex Labs:**
- 5-service dependency chain
- Simulate real prod topology
- Inject various failure modes
- Practice debugging

Week 7-8 Deliverable: Trace-instrumented multi-service app with capacity analysis

WEEK 9-10: SLO/Error Budget Practice

Goal: Advanced SLO topics and stakeholder negotiation

Day 57-60: Multi-Service SLOs

- Calculate composite SLOs
- Understand SLO composition math
- Design per-service SLO targets

Day 61-64: Error Budget Negotiation

- Practice scenarios:
- Product wants to launch, budget low
- Multiple launches needed
- Post-incident recovery

Day 65-70: Real-World SLO Design

- Design SLOs for complex services:
- Multi-endpoint services
- Read vs write paths
- Critical vs non-critical operations

Week 9-10 Deliverable: SLO proposals for 3 different service types

WEEK 11-12: Cloud Providers & Advanced Scenarios

Goal: Understand cloud-specific failure modes and observability

Day 71-75: AWS Deep Dive

- Set up:
 - CloudWatch metrics
 - ELB health checks
 - RDS failure modes
 - EC2 instance types
- **Lab:** Deploy app on AWS, observe cloud-specific metrics

Day 76-80: GCP & Azure Comparison

- Deploy same app on GCP and Azure
- Compare:
- Metrics APIs
- Load balancer behavior
- Database options
- Failure isolation

Day 81-84: Cloud Failure Modes

- Simulate:
- AZ failure
- Instance retirement
- Managed service degradation
- Quota exhaustion

Week 11-12 Deliverable: Cloud comparison matrix and cross-cloud incident response runbook

WEEK 13: Interview Preparation & Gap Analysis

Goal: Identify gaps and prepare for interviews

Day 85-87: Mock Interviews

- Phone screen scenarios
- System design questions
- Incident response walkthroughs

Day 88-89: Portfolio Review

- Review all deliverables
- Identify weak areas
- Additional practice on weak areas

Day 90: Final Assessment

- **Comprehensive Scenario** (4 hours):
- Deploy unfamiliar multi-service app

- Set up full observability
- Define SLIs/SLOs
- Simulate 3 different incident types
- Detect, mitigate, document all
- Present findings

Success Criteria for Day 90:

- Complete observability setup in < 1 hour
 - Detect all incidents in < 5 minutes
 - Mitigate all incidents in < 10 minutes
 - Write clear post-mortems
 - Answer interviewer questions about decisions
-

Resources (Free/Low-Cost)

Books:

- Google SRE Book (free online)
- Google SRE Workbook (free online)
- "Designing Data-Intensive Applications" (library/ebook)

Tools:

- Docker Desktop (free)
- AWS Free Tier
- GCP \$300 credit
- Azure Free Tier

Practice Environments:

- Docker Compose for local setups
- Free-tier cloud accounts
- Open-source sample applications

Communities:

- r/sre (Reddit)
 - SRE Weekly newsletter
 - Chaos Engineering Slack
-

Daily Time Investment

Weeks 1-2 (14-day sprint): 4 hours/day (28 hours/week)

Weeks 3-10: 2-3 hours/day (15-20 hours/week)

Weeks 11-13: 3-4 hours/day (20-25 hours/week)

Total: ~200 hours over 90 days

Failure Checks (Self-Assessment)

After each week, answer:

1. Can I explain this concept to someone else?
2. Can I debug a failure using this knowledge?
3. Can I answer interview questions on this topic?

If NO to any: Spend extra time on that topic before proceeding.

Expected Capabilities by Week

Week 2: Phone screen ready (observability basics)

Week 4: Can discuss incidents in interviews

Week 6: Can explain reliability patterns

Week 8: Can handle distributed systems questions

Week 10: Can discuss SLO negotiation with product

Week 12: Can compare cloud providers

Week 13: Ready for on-site interviews

What This Doesn't Cover (On-the-Job Learning)

- Advanced Kubernetes operations
- Chaos engineering frameworks (Chaos Mesh, Gremlin)
- Multi-region failover orchestration
- Security SRE specifics
- Cost optimization at scale

- Team/org SRE practices

These require production environment access and are learned on the job after you're hired.